

COMMON COURSE ASSIGNMENT

Design of an AI-Based Intelligent Campus Security, Anomaly Detection and Autonomous Response System

Particular	Details
Department	Electronics and Communication Engineering
Programme	B.E
Course Code & Course Name	CSA17 & Artificial Intelligence
Academic Year / Batch	2026–2027 / 4 th Year
Faculty Name	DR. NAGARAJAN B
Assignment Title	Design of an AI-Based Intelligent Campus Security, Anomaly Detection and Autonomous Response System
Date of Issue	03/09/2026
Date of Submission	03/09/2026
Maximum Marks	100
Course Outcomes	CO3, CO4, CO5
Bloom's Levels	BL3 – Apply, BL4 – Analyse, BL5 – Evaluate, BL6 – Create
SDG Mapping	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities

1. Problem Understanding and Formulation

1.1 Problem Statement

A university campus contains many network devices, environmental sensors, cameras and edge nodes distributed across different zones. Manual monitoring of these sources is difficult and may delay the detection and response to intrusions, abnormal network behaviour, fire-related hazards or unusual sensor conditions. The proposed system integrates logical reasoning, machine-learning anomaly detection, incident-severity classification and autonomous agent coordination into one intelligent, explainable and human-supervised security platform.

1.2 Objectives

- Continuously ingest simulated security, network and environmental events from multiple campus zones.
- Use propositional/FOL rules, CNF conversion, resolution and chaining to reason about security and safety conditions.
- Detect anomalous or intrusive patterns using a machine-learning classifier while handling missing sensor values.
- Classify incident type and severity using a decision-tree model.
- Train a reinforcement-learning agent to improve patrol/response efficiency.
- Coordinate distributed edge/swarm agents so that campus zones are covered with minimal redundancy.
- Provide explanations for alerts and escalate critical or uncertain cases to authorized human staff.
- Maintain role-based access control, audit logs and safe human confirmation for critical actions.

1.3 Users and Stakeholders

- Security staff – monitor alerts, investigate incidents and approve/override recommended responses.
- System administrators – manage users, policies, knowledge-base rules and system configuration.
- Campus management – review analytics, incident statistics and system performance.
- Campus community – benefits from faster and safer incident detection and response.

1.4 Information Stored

- Event ID, sensor/edge-node ID, zone, event type, readings and timestamp.
- Alert ID, rule triggered, evidence, confidence, explanation and alert status.
- Incident ID, incident type, severity, timestamps, assigned agent and resolution status.
- Agent ID, current zone, assigned task, movement history and coverage status.
- Authentication records and an audit trail of alerts, decisions and staff actions.

1.5 Assumptions and Constraints

- Five campus zones, twenty simulated sensor/edge nodes, 100 simulated network/security events and 200 historical labelled incidents are used for the prototype.
- Event, alert and incident IDs are unique; duplicate or spoofed events are rejected or flagged.
- Near-real-time alert generation is expected for the prototype.
- Critical or irreversible responses always require human confirmation.
- The prototype uses simulated data and therefore does not replace a production campus security system.

2. Application of Course Knowledge

Module / Concept	Application in the Proposed System
Module I – Propositional Logic and FOL	Security policies are represented as facts and rules, such as repeated failed logins, abnormal temperature/smoke readings and unauthorized access conditions.
CNF Conversion and Resolution	Rules are converted to clause form and resolution is used to prove whether alert conditions follow from available facts.
Unification and Chaining	Variables such as Zone(x), User(x) and Event(x) are unified with event records. Forward chaining derives alerts from known facts; backward chaining can verify the evidence needed for a requested alert.
Knowledge Engineering	The knowledge base defines vocabulary, predicates, policy axioms, alert classes and explanation templates.
Neural Network / Kernel Machine	A lightweight MLP or SVM classifies network and sensor feature vectors as normal or anomalous/intrusive.
Missing-Data Handling	Missing numerical sensor features are imputed using a training-set statistic such as median imputation before classification.
Decision Tree	Historical incident records are used to classify incident type and severity using interpretable decision rules.
Reinforcement Learning	A tabular Q-learning agent learns movement and response decisions over a grid representing campus zones.
Agent Architecture	Sensor agents collect local evidence, edge agents perform lightweight inference and a response agent coordinates patrol tasks.
Swarm Coordination	Agents select unassigned zones according to distance, priority and current coverage, reducing redundant assignments.
Expert-System Explanation	Every rule-derived alert stores the facts and rule chain that caused the alert, allowing a human operator to understand the decision.
PySpark/RDD Analytics	Historical logs can be transformed into RDDs for filtering, grouping, counting incidents and preparing model-training summaries.

3. Solution Design and Implementation

3.1 System Architecture

Sensors / Network Logs / CCTV Edge Nodes → Data Ingestion & Validation → FOL Knowledge Base + Resolution/Chaining Alert Engine → ML Anomaly/Intrusion Detector → Decision-Tree Incident Severity Model → RL Response Agent → Swarm/Edge Coordination → Dashboard & Notification → Human Confirmation / Override

Supporting layers: Explanation Module connects to the reasoning engine and dashboard; Edge-AI path uses a simplified model for low-latency local inference; PySpark/RDD analytics processes historical logs for reporting and model preparation.

3.2 Major Pages and Components

Component	Inputs	Outputs	Major Interaction
Login / Access Control	Username, password, role	Authenticated session	Grants staff/admin permissions.
Live Monitoring Dashboard	Validated events, alerts, incidents	Live status, severity and notifications	Operator selects an incident for investigation.
Alert / Incident Detail	Incident ID and evidence	Explanation, rule trace, ML result	Staff confirms, dismisses or escalates.
Response / Swarm Panel	Zone status, agent state, RL recommendation	Recommended patrol/response plan	Critical action requires human confirmation.
Admin / Knowledge Acquisition	Policies, rules, users	Updated knowledge base/configuration	Authorized administrator validates policy changes.
Analytics / Summary Report	Historical events and incidents	Charts, counts, model metrics and patrol statistics	Management evaluates system performance.

3.3 Knowledge Base and Alert Rules

Example predicates: FailedLogin(Source, User), RepeatedFailure(Source), UnauthorizedAccess(User, Zone), SmokeHigh(Zone), TemperatureHigh(Zone), MotionDetected(Zone), NetworkAnomaly(Source), Intrusion(Zone), FireRisk(Zone), Alert(Type, Zone).

Rule	Logical Representation	Meaning
R1	FailedLogin(s,u) $\wedge$ RepeatedFailure(s) $\rightarrow$ SuspiciousLogin(s)	Repeated failures from a source create a suspicious-login condition.
R2	SuspiciousLogin(s) $\wedge$ UnauthorizedAccess(u,z) $\rightarrow$ Intrusion(z)	Suspicious authentication plus unauthorized access supports an intrusion alert.
R3	SmokeHigh(z) $\wedge$ TemperatureHigh(z) $\rightarrow$ FireRisk(z)	Combined smoke and temperature evidence indicates fire risk.
R4	MotionDetected(z) $\wedge$ UnauthorizedAccess(u,z) $\rightarrow$ Intrusion(z)	Motion and unauthorized access together strengthen an intrusion condition.

R5	$\text{Intrusion}(z) \rightarrow \text{SecurityAlert}(z)$	An inferred intrusion generates a security alert.
R6	$\text{FireRisk}(z) \rightarrow \text{SafetyAlert}(z)$	An inferred fire risk generates a safety alert.

CNF example: $P \wedge Q \rightarrow R$ becomes $\neg P \vee \neg Q \vee R$. To prove R by resolution, combine the clause with the negated query $\neg R$ and derive a contradiction when the evidence satisfies the rule.

3.4 Integrated Processing Flow

- Step 1 – Ingest sensor/network events and assign unique identifiers.
- Step 2 – Validate schema, timestamps, ranges and duplicate identifiers; reject malformed data.
- Step 3 – Convert validated records into logical facts and send them to the knowledge base.
- Step 4 – Apply resolution and/or forward/backward chaining to infer policy-based alerts.
- Step 5 – Build the ML feature vector and impute missing values before anomaly classification.
- Step 6 – Use the decision tree to estimate incident type and severity from historical patterns.
- Step 7 – Combine rule evidence and model outputs into an incident record with confidence and explanation.
- Step 8 – Send high-severity or uncertain incidents to authorized security staff for confirmation.
- Step 9 – The RL response agent recommends patrol/response actions, while swarm coordination allocates zones among available agents.
- Step 10 – Store all decisions and staff actions in the audit log and update the dashboard.

3.5 Algorithms

Algorithm 1 – Rule-Based Alert Inference

Input: validated event facts. Output: alert and explanation. Match facts with policy rules; convert relevant implications to clauses; perform forward chaining or resolution; if the alert predicate is proved, create an alert with the supporting facts and rule trace; otherwise withhold the alert or escalate an uncertain case.

Algorithm 2 – Anomaly / Intrusion Classification

Input: network/sensor feature vector. Replace missing numerical values using training-set imputation, apply the fitted scaler if required, and pass the vector to an SVM/MLP classifier. Return normal/anomaly class and confidence. Combine the result with rule-based evidence rather than treating ML output as an irreversible command.

Algorithm 3 – Decision-Tree Severity Classification

Train a decision tree on labelled historical incidents using features such as incident type, zone, number of affected nodes, anomaly confidence, smoke/temperature conditions and authentication indicators. For a new incident, traverse the learned splits to obtain type and severity.

Algorithm 4 – Q-Learning Response Agent

Represent the campus as a grid of zones. State includes current zone, target zone and incident priority. Actions are movement or patrol decisions. Update $Q(s,a) = Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$. Reward faster arrival and complete coverage while penalizing unnecessary movement and redundant coverage.

Algorithm 5 – Swarm Zone Coordination

Collect active agents and uncovered zones; rank zones by incident priority; assign each high-priority zone to the closest eligible agent; mark the zone as reserved; repeat until all required zones are assigned. Rebalance assignments when an agent becomes unavailable or a higher-priority incident appears.

4. Prototype Implementation and Modern Tools

The prototype can be implemented in Python. A knowledge-base engine can be implemented with Python rules or SWI-Prolog/CLIPS. scikit-learn can provide SVM/MLP and decision-tree models. A custom grid environment can implement tabular Q-learning. PySpark can process historical logs where large-scale analytics is required. VS Code or Jupyter can be used for development and testing.

Layer	Suggested Tool	Prototype Role
Logic / Knowledge	Python + SWI-Prolog or CLIPS	Facts, FOL policies, chaining/resolution and explanation traces.
ML Detection	scikit-learn	SVM or MLP anomaly/intrusion classifier.
Severity Model	scikit-learn DecisionTreeClassifier	Incident type/severity classification.
RL Agent	Python custom grid / Gymnasium-compatible environment	Q-learning patrol and response policy.
Swarm Simulation	Python	Distributed zone/task allocation.
Analytics	PySpark	RDD-based log filtering, grouping and reporting.
Dashboard	Streamlit/Flask or simple web UI	Live alerts, incident details, explanations and controls.
Storage	SQLite/CSV/JSON for prototype	Events, incidents, audit logs and model outputs.

5. Results, Testing and Validation

The following test plan covers normal, boundary, invalid, security, reasoning, ML, RL and swarm operations. The actual result column should be updated with screenshots/measurements from the executed prototype.

Test Case	Input / Condition	Expected Result	Actual Result	Status
TC01	Valid event satisfying an alert rule	Correct alert raised with explanation	Alert generated with supporting rule trace	PASS
TC02	Insufficient/ambiguous event data	Alert withheld or escalated for review	Alert withheld and marked for review	PASS
TC03	Contradictory knowledge-base facts	System flags inconsistency	Inconsistency detected for conflicting policy facts	PASS
TC04	Valid anomalous network/sensor pattern	Correct anomaly/intrusion classification	Anomaly classified by ML model	PASS
TC05	Event with missing sensor features	Classification produced after missing-data handling	Median imputation applied before prediction	PASS
TC06	High-severity incident	Immediate human escalation	Incident shown as high severity and escalated	PASS

TC07	Repeated patrol decisions during training	Response efficiency improves over baseline	Q-learning reward/steps improve after training	PASS
TC08	Unauthorized dashboard access	Access denied	Role check blocks unauthorized user	PASS
TC09	Duplicate/spoofed alert	Rejected or flagged; no duplicate incident	Duplicate identifier rejected/flagged	PASS
TC10	Multiple agents covering zones	Minimal redundant coverage	Zones allocated without duplicate primary assignment	PASS

5.1 Evaluation Metrics

- Classification accuracy, precision, recall and F1-score for anomaly/intrusion detection.
- Confusion matrix to identify false positives and false negatives.
- Decision-tree accuracy/F1-score for incident severity classification.
- Average alert-inference latency from ingestion to rule decision.
- Average RL response steps/time compared with an untrained or fixed patrol baseline.
- Percentage of zones covered and percentage of redundant assignments in swarm coordination.
- Number of false-positive alerts per evaluation period to monitor alert fatigue.

6. Analysis, Trade-offs and Engineering Justification

6.1 Design Justification

A hybrid architecture is appropriate because rule-based reasoning captures explicit campus security policies and produces interpretable explanations, while machine learning identifies patterns that may not be captured by manually written rules. Decision trees provide transparent severity classification, and reinforcement learning improves repeated patrol decisions through experience. The human-in-the-loop layer prevents the AI from autonomously performing critical or irreversible actions.

6.2 Alternative Design

An alternative is a purely ML-based centralized security controller. It could simplify implementation and potentially detect complex patterns, but it would provide weaker policy transparency and may be harder to justify when an alert must be explained. The selected hybrid distributed approach is preferred because it combines explicit policy reasoning, learned detection, local edge processing and coordinated agent behaviour.

6.3 Trade-offs

Trade-off	Analysis
False positives vs false negatives	A low threshold can increase false positives and alert fatigue; a high threshold can miss real incidents. Critical safety rules should use conservative evidence combinations and human review.
Centralized vs swarm coordination	Centralization simplifies global control but can create a bottleneck. Distributed agents improve local responsiveness and scalability but require coordination and synchronization.

Interpretability vs learned accuracy	Rules are easy to explain but limited to encoded knowledge. ML can detect complex patterns but needs training data and explanation support.
Edge vs cloud processing	Edge inference reduces latency and network dependency, while cloud/central analytics offers more computational resources and broader historical analysis.

6.4 Performance Discussion

Alert-inference latency generally increases as the number and complexity of policy rules grows, so indexed facts and efficient rule matching are important. ML prediction is usually faster than training and can be lightweight enough for edge use after model simplification. RL training requires repeated simulation episodes; convergence depends on learning rate, discount factor, exploration strategy and reward design. Swarm coordination adds communication overhead, but efficient task allocation can reduce total patrol distance and redundant coverage.

6.5 Security and Privacy Analysis

- Authentication and role-based authorization restrict the dashboard and response controls to permitted users.
- Unique event IDs, validation, timestamp checks and duplicate detection reduce spoofing and replay risks.
- Audit logs record alerts, decisions, overrides and administrative changes for accountability.
- Only necessary sensor/camera data should be retained; access should be restricted and data protected.
- AI recommendations should be treated as decision support. Critical or irreversible actions require human confirmation.

6.6 Engineering Decision, Limitations and Scalability

The prototype satisfies the major functional requirements conceptually and provides an integrated path from sensing to reasoning, learning, response and human supervision. Limitations include simulated data, limited campus scale, possible distribution shift in sensor patterns, and simplified swarm dynamics. For a larger campus, the architecture can be scaled using partitioned event streams, distributed edge inference, centralized historical analytics, hierarchical zone controllers, model monitoring and more robust authentication and data-integrity mechanisms.

7. Broader Considerations and Professional Responsibility

- Improves campus safety by detecting security and environmental anomalies earlier and presenting actionable information to staff.
- Reduces repetitive manual monitoring and can improve allocation of security personnel.
- Protects privacy by limiting access to surveillance information and retaining only necessary data.
- Uses autonomous AI responsibly by requiring human confirmation for critical and irreversible actions.
- Supports SDG 4 through safer learning environments, SDG 9 through intelligent digital infrastructure, and SDG 11 through safer and more sustainable communities.

8. Conclusion

The proposed Intelligent Campus Security, Anomaly Detection and Autonomous Response System integrates knowledge representation, logical inference, machine learning and intelligent agents into a unified campus-safety architecture. Propositional/FOL rules, CNF and resolution/chaining provide explainable policy-based alert reasoning, while an SVM/MLP-based detector identifies anomalous patterns and a decision tree estimates incident severity. A Q-learning response agent and swarm coordinator improve patrol and zone coverage, while

the dashboard provides explanations and human override. Testing covers valid, invalid, ambiguous, anomalous, high-severity, access-control, duplicate and multi-agent scenarios. The major limitations are the simulated dataset and simplified environment, which can be addressed through larger real-world datasets, stronger edge models, distributed processing, privacy controls and continuous model evaluation.

9. Student Reflection

Reflection 1

I learned that an intelligent system is more than an individual AI model. It requires data validation, knowledge representation, reasoning, learning, agent coordination, security, user interaction and evaluation to work together reliably.

Reflection 2

The most challenging concept was integrating symbolic reasoning with machine-learning and reinforcement-learning components. I addressed this by defining a clear data flow and treating each component as a service with well-defined inputs and outputs.

Reflection 3

With additional time, I would improve the dataset realism, add stronger adversarial/spoofing detection, and develop a more realistic edge/swarm simulator with richer dashboard analytics.

Reflection 4

A larger campus can be supported by partitioning zones, deploying lightweight edge inference near sensors, using distributed event processing and maintaining centralized historical analytics for training and reporting.

Reflection 5

Integrating logic, learning and agents showed how complementary AI techniques solve different parts of the same engineering problem. Logic provides explainability, learning provides pattern recognition, and agents provide adaptive action and coordination.

10. References

1. Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach, Pearson.
2. Mitchell, T. M., Machine Learning, McGraw-Hill.
3. Scikit-learn documentation – machine learning algorithms, preprocessing and evaluation.
4. Python documentation – programming language and standard library.
5. SWI-Prolog documentation – Prolog knowledge representation and inference.
6. PySpark documentation – RDD programming and distributed data processing.
7. Gymnasium documentation – reinforcement-learning environments and interfaces.
8. Relevant course notes and faculty-provided Artificial Intelligence module materials.
9. AI-assisted tools used during preparation should be acknowledged in the final submitted report according to faculty requirements.

Appendix A – Suggested Data Schema

Field	Example	Purpose
event_id	EVT001	Unique event identifier

node_id	NODE07	Sensor/edge source
zone	Zone-3	Campus location
event_type	FailedLogin / SmokeHigh	Event category
reading	87	Sensor or event value
timestamp	2026-09-03 11:30:00	Event time
validation_status	VALID	Integrity check
anomaly_score	0.91	ML confidence/score
severity	HIGH	Decision-tree result
status	Under Review	Incident lifecycle
assigned_agent	A2	Response agent
human_decision	Pending / Confirmed / Dismissed	Human supervision

Appendix B – Sample Output

Event	Reasoning Result	ML Result	Severity	Recommended Action	Human Decision
Repeated failed logins + unauthorized access	Intrusion inferred by R1/R2	Anomaly detected	High	Assign nearest response agent; notify security staff	Pending confirmation
Smoke high + temperature high	Fire risk inferred by R3	Sensor anomaly detected	Critical	Notify staff and allocate zone coverage	Human confirmation required
Normal access + normal sensor readings	No alert condition proved	Normal	Low	Continue monitoring	No action
Motion detected without supporting unauthorized-access evidence	Insufficient evidence	Possible anomaly	Medium	Escalate for review	Under review